



Trailhead



[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)



Communicate with Events ▾

Communicate with Events

Learning Objectives

After completing this unit, you'll be able to:

- Describe differences in events for the two programming models.
- Send an event from a Lightning web component to an Aura component.

A component walled off by itself can be useful, but the real app magic happens when components start talking to each other. That's where events come in. Using events to communicate between components is a web standard, and events are central to communication in both Aura components and Lightning web components.

Component Events Become DOM Events

Migrate component events in Aura components to web-standard DOM events in Lightning web components. If you're familiar with DOM events, you notice the same propagation behavior in Lightning web components.

Create and Dispatch an Event

Instead of the proprietary Event object in Aura components, use the Event or CustomEvent standard DOM interfaces. We strongly recommend you use the CustomEvent interface, because this provides a more consistent experience. The framework may need to patch properties and methods of events to normalize implementations between browsers, and using CustomEvent guarantees that your component is covered. Internet Explorer doesn't support the CustomEvent interface natively, but the Lightning Web Components model adds support for it.

There's no equivalent in Lightning web components for the <aura:registerEvent> tag in Aura component markup to register that a component can fire an event. Thanks to using



Hi NELLURI, ask me about products, features, and pricing, or connect to a sale...





```
1 export default class PropertyTile extends LightningElement {  
2     @api property;  
3     propertySelected() {  
4         const selectedEvent = new CustomEvent('selected', {  
5             detail: this.property.Id,  
6         });  
7         this.dispatchEvent(selectedEvent);  
8     }  
9 }
```

Communicate with Events

The first argument in the CustomEvent() constructor sets the name of the event to selected.

The second argument is an object that configures the event behavior. In this object, we set the detail, which is the data payload for the event. A handling component can read the data. In this case, we're passing the this.property.Id value.

Handle an Event

An Aura component uses the <aura:handler> tag in markup to define a handler. Alternatively, a component can declare a handler action when it references another component in its markup.

This Aura component uses c:child in its markup and declares a handleNotification handler for the notification event that c:child fires.

```
1 <c:child notification="{!c.handleNotification}"/>
```

A Lightning web component can similarly have a declarative handler. The event name in the declarative handler is prefixed by on.

```
1 <c-child onnotification={handleNotification}></c-child>
```

The handleNotification event handler function is defined in the JavaScript file of the





web component handles this event in its HTML file.

Lightning Web Components for Aura Developers (/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

```
1 <template for:each={properties} for:item="property">
2   <c-property-tile property={property} key={property.id}
3   onselected={onPropertySelected}></c-property-tile>
</template>
```

Communicate with Events

The selected event name is prefixed by on in the declarative handler configured by onselected={onPropertySelected}.

The onPropertySelected event handler function is defined in propertyTileList.js.

```
1 onPropertySelected(event) {
2   pubsub.fire('propertySelected', event.detail);
3 }
```

Cast your mind back to when we fired the selected event in propertyTile. The CustomEvent() constructor set detail to this.property.Id. The handler unpacks the event data by accessing event.detail. In this example, we get the property ID in event.detail.

We explain what's going on with pubsub.fire() when we look at application events next.

Application Events Become a Publish-Subscribe Pattern

Migrate application events in Aura components to a pub-sub (publish-subscribe) pattern in Lightning web components. In a pub-sub pattern, one component publishes an event and other components subscribe to receive and handle the event. Every component that subscribes to the event receives the event.

Standard DOM events should always be your first choice for events because they only travel up the containment hierarchy, making behavior more predictable. Application events can be problematic in complex apps because any component can handle the event. This pattern can lead to code that is hard to maintain due to unexpected coupling of components. However, sometimes you need sibling components in a containment hierarchy to communicate. Unrelated components used in Lightning pages or the





register Registers a callback for an event. unregister Unregisters a callback for an event.
fire Fires an event to listeners.

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=sreballo78&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=sreballo78&trailmix_slug=salesforce-platform-developer)

Sending Events to an Enclosing Aura Component

Communicating with Events
Lightning web components dispatch DOM events. An enclosing Aura component can listen for these events, just like an enclosing Lightning web component can. The enclosing Aura component can capture the event and handle it. Optionally, the Aura component can fire an Aura event to communicate with other Aura components or with the app container.

This technique is useful when you want to support events or interfaces that are used in Aura components but not currently supported by Lightning web components.

The following example illustrates this technique. This Lightning web component fires a custom notification event in its JavaScript file.

```
1 // catchAndRelease.js
2 import { LightningElement } from 'lwc';
3 export default class CatchAndRelease extends LightningElement {
4     /**
5      * Handler for 'Fire My Toast' button.
6      * @param {Event} evt click event.
7      */
8     handleFireMyToast(evt) {
9         const eventName = 'notification';
10        const event = new CustomEvent(eventName, {
11            detail: { message: 'See you on the other side.' }
12        });
13        this.dispatchEvent(event);
14    }
15 }
```



The enclosing Aura component wrapper adds a handler for the custom event. Notice that the event handler, onnotification, matches the event name with on prefixed to it. That is, use onnotification to handle the event named notification.





component that the DOM event bubbles to.

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers/custom-event-functions-in-the-78-trailmix-for-aura-developers\)](#) Aura component handles the event.

Communicate with Events

```
1 // catchAndReleaseWrapperController.js
2 ({
3     handleCustomEvent: function(cmp, evt) {
4         // Get details from the DOM event fired by the Lightning
5         web component
6         var msg = evt.getParam('message') || '';
7     }
8 })
```



The notification event dispatched by the Lightning web component set the event detail.

```
1 detail: { message: 'See you on the other side.' }
```



The Aura component that handles the event accesses the event data using `evt.getParam('message')`.

You can handle the event however you'd like. You can optionally fire a new Aura event to communicate with other Aura components.

Resources

- [Lightning Aura Components Developer Guide: Communicating with Events](https://developer.salesforce.com/docs/atlas.en-us.lightning.meta/lightning/events_intro.htm) (https://developer.salesforce.com/docs/atlas.en-us.lightning.meta/lightning/events_intro.htm)
- [Lightning Web Components Developer Guide: Migrate Events](https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.migrate_events) (https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.migrate_events)





- Explore the [DreamHouse app code](https://github.com/trailheadapps/dreamhouse-lwc) (<https://github.com/trailheadapps/dreamhouse-lwc>) to see how all the Lightning Web Components for Aura Developers (https://developer.salesforce.com/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer) components fit together.
- Check out the [lwc-recipes](https://github.com/trailheadapps/lwc-recipes) (<https://github.com/trailheadapps/lwc-recipes>) repo on [GitHub](#) with [Events](#) of tasty code samples for common use cases.
- See more sample apps in the [Sample Gallery](https://trailhead.salesforce.com/sample-gallery) (<https://trailhead.salesforce.com/sample-gallery>).

Even if you hit a few traffic jams or unexpected diversions along the way, we hope you enjoy the journey!

But What About...?

Unfortunately, there's a limit to the amount of material that we can cover in one Trailhead module. A module is like a bucket of water. There's only so much water that you can pour in before it starts to overflow. If we opt for a bigger bucket, it becomes too heavy to carry around.

If we didn't cover a feature that you're interested in, it's not because we didn't want to. The bucket is simply full.

Testing Components

In particular, we didn't cover the important area of component testing. Here are some pointers to further information in the developer guides.

In Aura components, use [Lightning Testing Service](https://developer.salesforce.com/docs/atlas.en-us.lightning.meta/lightning/lightning_testing_intro.htm) (https://developer.salesforce.com/docs/atlas.en-us.lightning.meta/lightning/lightning_testing_intro.htm) to test components.

In Lightning web components, use Jest for unit testing. If Jest doesn't cover all your test cases, use Lightning Testing Service to cover the remaining test cases. See [Test Lightning Web Components](https://developer.salesforce.com/docs/platform/lwc/guide/testing.html) (<https://developer.salesforce.com/docs/platform/lwc/guide/testing.html>).

Aura Methods Become JavaScript Methods





(https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.create_javascript_methods) in the Lightning Web Components Developer Guide (/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer).

Migrate Interfaces

The primary usage of interfaces in Aura components is for marker interfaces, which enable specific usage for a component in an app. A marker interface is an empty interface with no attributes, events, or methods.

In Lightning web components, usage is defined in a component's configuration file or with some simple `@api` decorators.

For more details, see [Migrate Interfaces](https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.migrate_interfaces) (https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.migrate_interfaces) in the Lightning Web Components Developer Guide.

Migrate Access Controls

To manage access to an Aura bundle's resources, such as a component or attribute, use the access system attribute.

The mechanism for access control is different in a Lightning web component, where access is defined by JavaScript property decorators and the component's configuration file.

For more details, see [Migrate Access Controls](https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.migrate_access_controls) (https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.migrate_access_controls) in the Lightning Web Components Developer Guide.

Quiz

To complete this unit, you need to answer all the quiz questions correctly.

+100 Points





A A JavaScript method

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

B A DOM event

Communicate with Events

C An API call

D An Apex event

2 What prefix does an Aura handler add to the event name of a Lightning web component?

A aura

B in

C on

D gotcha

Second attempt earns 50 points. Three or more earns 25 points.

Check the Quiz to Earn 100 Points





Trails	Maintain Certifications	Events	Sales Enablement
Trailmixes	Verify Certifications	Agentblazers	Trailblazer Store
<u>Lightning Web Components for Aura Developers (/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)</u>			
Superbadges		Salesforce Developers	Salesforce Help
Trailhead Academy		Salesforce Admins	
<u>Communicate with Events</u>		Code of Conduct	
Career Paths		Report Illegal Content (EU)	

Download Trailhead GO



© 2026 Salesforce, Inc. All rights reserved.

[Privacy Statement](#) [Terms of Use](#) [Use of Cookies](#) [Trust](#) [Accessibility](#) [Cookie Preferences](#)

Engli

Your Privacy Choices

